COMPUTATIONAL PHOTOGRAPHY · FALL 2026

Fun with Filters and Frequencies

Moksh Trehan · Assignment 2 · Updated October 19, 2026

Convolution isolates edges, separates image detail by spatial scale, and combines photographs in ways that change with viewing distance. This report follows those connections through explicit implementations, numerical checks, and visual experiments.

Included

NumPy convolution · Finite differences · Derivatives of Gaussians · HSV gradient orientations · Unsharp masking · Three hybrid images · Gaussian and Laplacian stacks · Oraple and two custom blends · Mandatory color experiments

Online report: https://mokshtrehan.github.io/syde671-assignment2/

Contents

1.1 · Convolutions from scratch

1.2 · Finite difference operator

1.3 · Derivative of Gaussian

Mandatory extension · Gradient orientations

2.1 · Image sharpening

2.2 · Hybrid images

Color · Which frequency band benefits?

2.3 · Gaussian and Laplacian stacks

2.4 · Multiresolution blending

Two custom color blends · Irregular selections

What I learned

Reproduction and image credits

1.1 · Convolutions from scratch

Both implementations compute true convolution: flip the kernel in both dimensions, zero-pad the image, and accumulate products. The four-loop version iterates over image rows, image columns, kernel rows, and kernel columns. The two-loop version keeps the two kernel loops and multiplies a whole shifted image slice at once. No convolution routine, FFT, window-view helper, or image-processing library is used inside either implementation.

For a kernel of height Kh and width Kw, padding before each axis is floor(K/2) and padding after it is floor((K–1)/2). This asymmetry for even kernels matches SciPy's "same" crop convention. Testing non-symmetric odd and even kernels is necessary: a symmetric Gaussian alone would not catch an accidental correlation or a one-pixel offset.

The grayscale selfie is 384 × 512 pixels; luminance uses 0.299R + 0.587G + 0.114B on RGB values in [0,1]. The 9 × 9 average removes fine texture. D_x and D_Y emphasize changes across columns and rows respectively; the signed display has one common symmetric color scale. Zero extension visibly darkens a smoothed outer rim and can introduce false derivatives at the frame.

For a fixed 96 × 96 selfie crop/resize, the table reports the median of three timed calls after one warm-up. Every method uses float64, zero extension, and "same" output. Maximum error across all comparisons is 8.88e-16. These are measurements on this computer, not universal speed claims. Both manual methods do O(HWKhKw) arithmetic; vectorizing the image operations removes Python's per-pixel overhead.

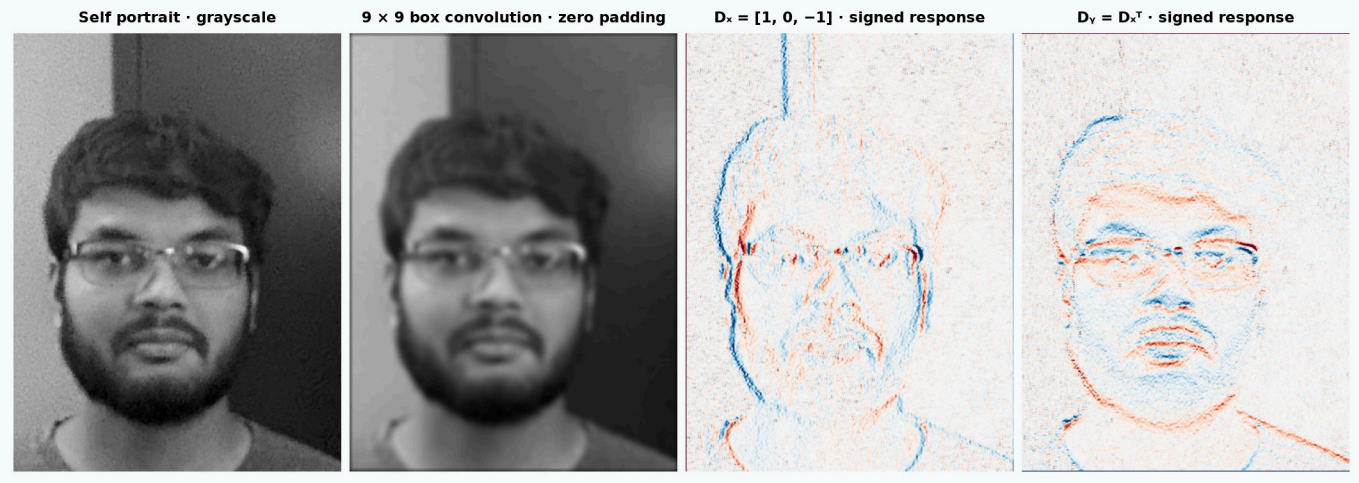
The boundary test uses an all-ones 25 × 25 image and a normalized 9 × 9 box. Zero fill produces 25/81 at a corner and 5/9 along an edge; symmetric and periodic extension preserve one everywhere. "Same" describes output size, not how pixels beyond the image are supplied.

Runtime and correctness (96 × 96 float64 input)

Kernel	Method	Median ms	Max |error|
3×5 asymmetric	Four loops	43.908	5.55e-16
3×5 asymmetric	Two loops	0.123	5.55e-16
3×5 asymmetric	SciPy convolve2d	0.209	0.00e+00
2×4 asymmetric (even)	Four loops	23.082	8.88e-16
2×4 asymmetric (even)	Two loops	0.08	8.88e-16
2×4 asymmetric (even)	SciPy convolve2d	0.138	0.00e+00
9×9 box	Four loops	209.111	7.77e-16
9×9 box	Two loops	0.609	7.77e-16
9×9 box	SciPy convolve2d	0.9	0.00e+00
1×3 derivative	Four loops	9.022	0.00e+00
1×3 derivative	Two loops	0.047	0.00e+00
1×3 derivative	SciPy convolve2d	0.095	0.00e+00

Boundary behavior of a constant image

SciPy boundary	Corner	Top edge midpoint	Center
fill	0.308642	0.555556	1.0
symm	1.0	1.0	1.0
wrap	1.0	1.0	1.0



Self portrait, 9 × 9 box-filtered portrait, and signed horizontal/vertical derivatives. Red is positive and blue is negative; shared derivative scale clips only the strongest responses for visibility.

Input validation, kernel flip, and exact padding

```
def _convolution_inputs(image, kernel):
    image = np.asarray(image, dtype=np.float64)
    kernel = np.asarray(kernel, dtype=np.float64)
    if image.ndim != 2 or kernel.ndim != 2:
        raise ValueError("image and kernel must both be two dimensional")
    if not image.size or not kernel.size:
        raise ValueError("image and kernel must be nonempty")
    kh, kw = kernel.shape
    # Before = ceil((K-1)/2); after = floor((K-1)/2).
    padded = np.pad(image, ((kh // 2, (kh - 1) // 2),
                            (kw // 2, (kw - 1) // 2)), mode='constant')
    return image, kernel[::-1, ::-1], padded
```

Four-loop convolution

```
def convolve_four_loops(image, kernel):
    """True 2-D convolution: two spatial loops and two kernel loops."""
    image, flipped, padded = _convolution_inputs(image, kernel)
    height, width = image.shape
    kh, kw = flipped.shape
    out = np.zeros_like(image)
    for y in range(height):
        for x in range(width):
            for ky in range(kh):
                for kx in range(kw):
                    out[y, x] += flipped[ky, kx] * padded[y + ky, x + kx]
    return out
```

Two-loop convolution

```
def convolve_two_loops(image, kernel):
    """True 2-D convolution: kernel loops; spatial products are vectorized."""
    image, flipped, padded = _convolution_inputs(image, kernel)
    height, width = image.shape
    kh, kw = flipped.shape
    out = np.zeros_like(image)
    for ky in range(kh):
        for kx in range(kw):
            out += flipped[ky, kx] * padded[ky:ky + height, kx:kx + width]
    return out
```

1.2 · Finite difference operator

The horizontal and vertical responses are $g_x = I * [1,0,-1]$ and $g_y = I * [1,0,-1]^T$. True convolution flips the stencil, so a left-to-right brightening gives positive g_x. Gradient magnitude is $\sqrt{(g_x^2 + g_y^2)}$; it combines both directions into an unsigned edge-strength map.

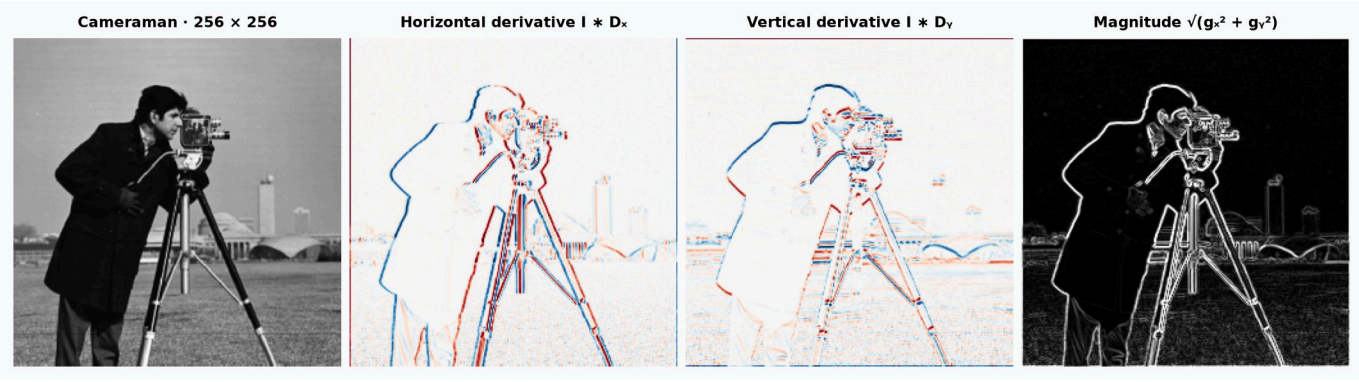
The supplied public cameraman rendering has a thin white border. I crop it to (1, 7, 535, 541) (left, top, right, bottom; right/bottom excluded), then resize the remaining square to 256 × 256 with Lanczos. All calculations use floating intensities in [0,1], with no per-image normalization before thresholding.

I compare thresholds 0.08, 0.16, and 0.24, which retain 23.7%, 12.8%, and 9.0% of the evaluated interior. I choose 0.16: it retains the person, camera, and tripod more clearly than 0.24 while rejecting much of the ground texture retained by 0.08. This is a visual tradeoff, not an accuracy score against annotated ground truth.

The edge predicate is magnitude > T. A seven-pixel frame is excluded from binary maps and edge-fraction measurements throughout Part 1, preventing zero-padding artifacts from being mistaken for scene edges and matching the largest Gaussian derivative support used below. The signed derivative and magnitude images still show their unmasked responses.

Threshold sensitivity

Threshold	Interior edge fraction	Selection
0.08	23.68%	Comparison
0.16	12.81%	Selected
0.24	8.99%	Comparison



Cameraman finite differences and gradient magnitude. Signed derivatives share [−0.65,0.65]; magnitude is displayed over [0,0.65].



Threshold sweep on the original finite-difference magnitude. The middle threshold balances contour continuity and background texture; black outer frame is excluded from evaluation.

Gradient and edge magnitude

```
gx = convolve_two_loops(image, DX)
gy = convolve_two_loops(image, DY)
magnitude = np.sqrt(gx * gx + gy * gy)
edges = magnitude > threshold
```

1.3 · Derivative of Gaussian

I construct a 13 × 13 Gaussian at σ = 2 pixels as the outer product of cv2.getGaussianKernel(13,2) with itself. First blur with G, then differentiate with D_x/D_Y. Equivalently, form Kx = G ∗ D_x and Ky = G ∗ D_Y once, then use a single convolution per derivative. These are finite-difference derivatives of the sampled Gaussian; their supports are 13 × 15 and 15 × 13 because full kernel convolution preserves the extra two samples.

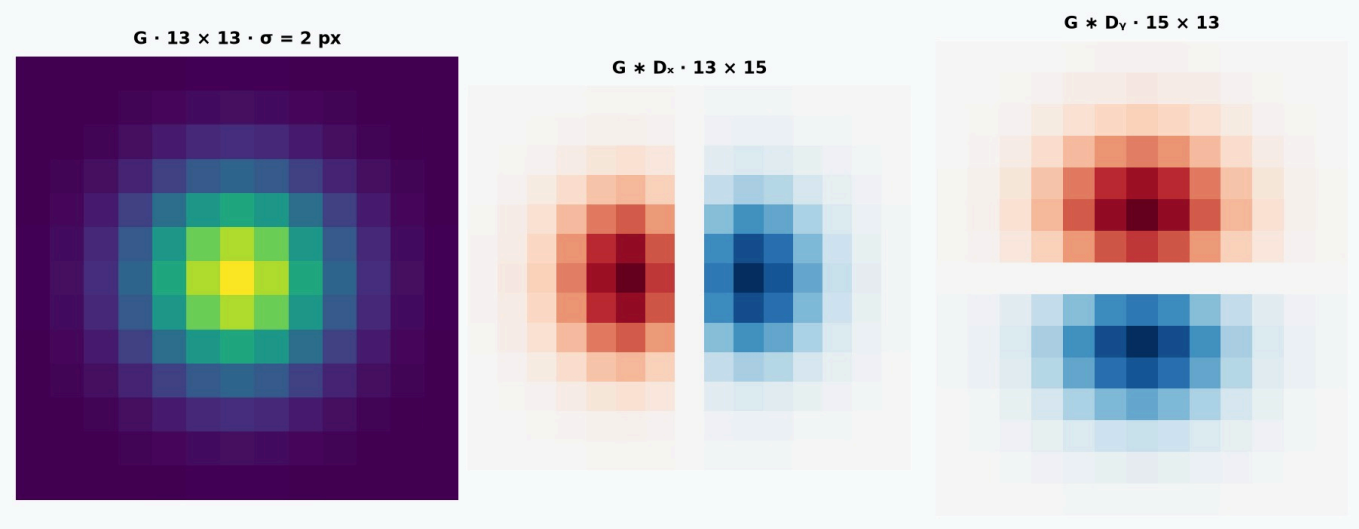
Convolution associativity gives I ∗ (G ∗ D) = (I ∗ G) ∗ D on the same zero-extended infinite image. In a finite implementation, calling “same” after BOTH stages throws away the blurred halo and breaks equality at the frame: the maximum error here is 0.307, although the evaluated interior agrees to 8.95e-16. Retaining the full Gaussian result, applying the derivative, and cropping only once restores agreement over the entire image to 8.95e-16. The discrepancy panel makes this boundary issue explicit.

Compared with raw finite differences, DoG removes fine grass texture and yields smoother, wider contours. It can merge nearby edges and remove weak detail; Gaussian smoothing is therefore a noise/detail tradeoff. I use T = 0.08 for DoG and T = 0.16 for raw gradients because smoothing lowers response amplitudes. These are separately chosen operating points, not proof that one detector has higher accuracy.

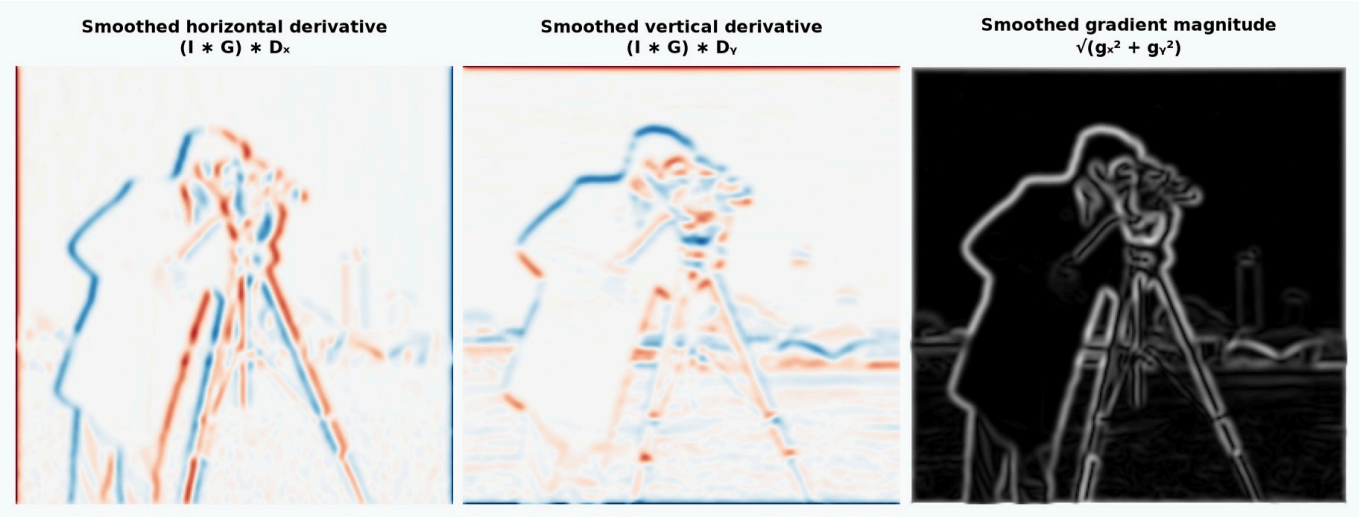
To test noise sensitivity, I add seeded Gaussian noise with σ = 0.08 and clip to [0,1]. I compare each noisy gradient with its own clean counterpart, avoiding the mistake of calling intentional smoothing a noise error. Interior vector-gradient residual RMS falls from 0.15226 for finite differences to 0.01030 for DoG (14.8× lower). This measures stability for this image/noise realization, not recovery of lost detail.

Convolution equivalence and noise measurements

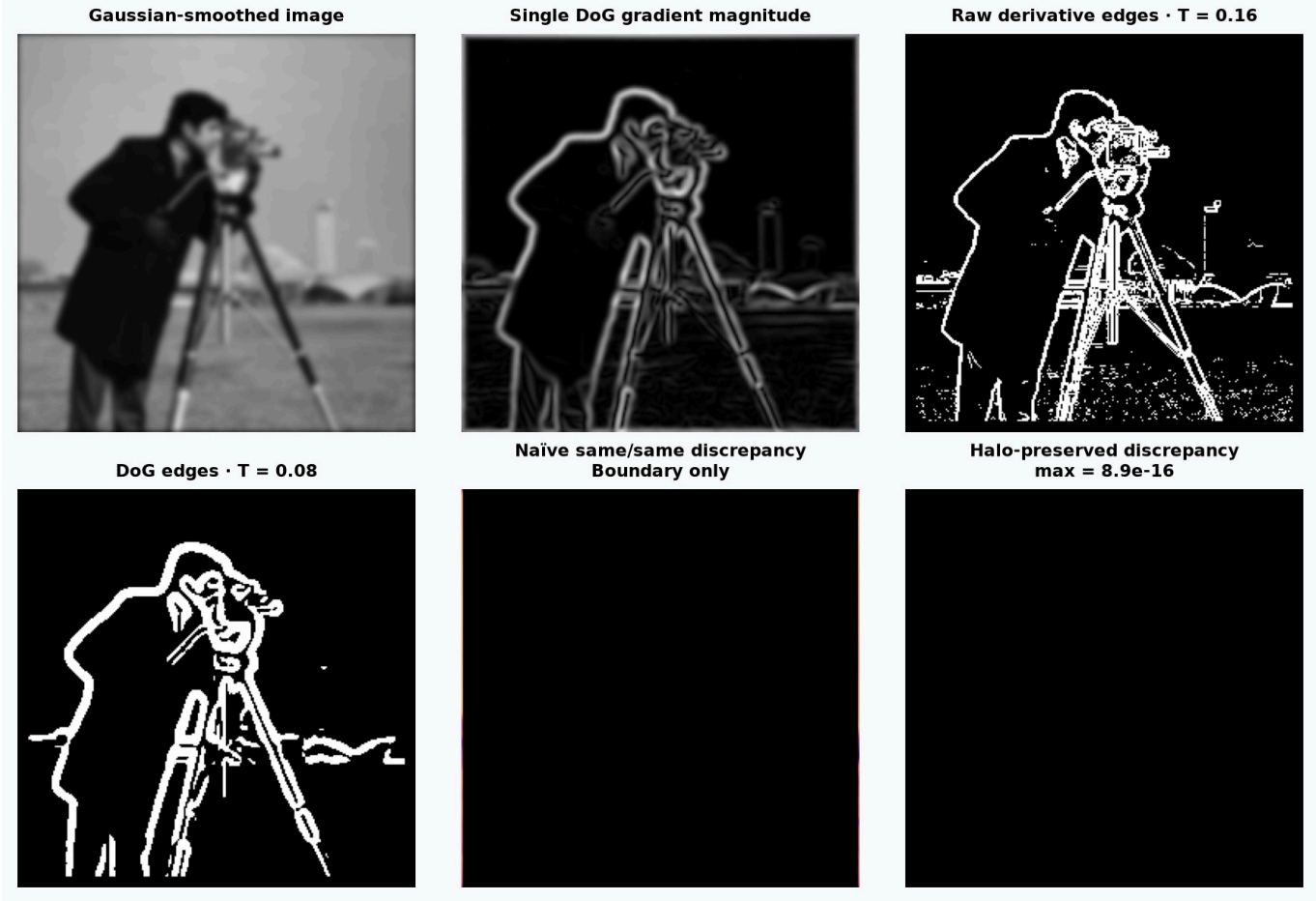
Measurement	Value
Same/same vs single DoG · max error	0.307441
Interior same/same vs single DoG · max error	8.95e-16
Halo-preserved sequential vs single DoG · max error	8.95e-16
Raw gradient noise RMS	0.152265
DoG gradient noise RMS	0.010304



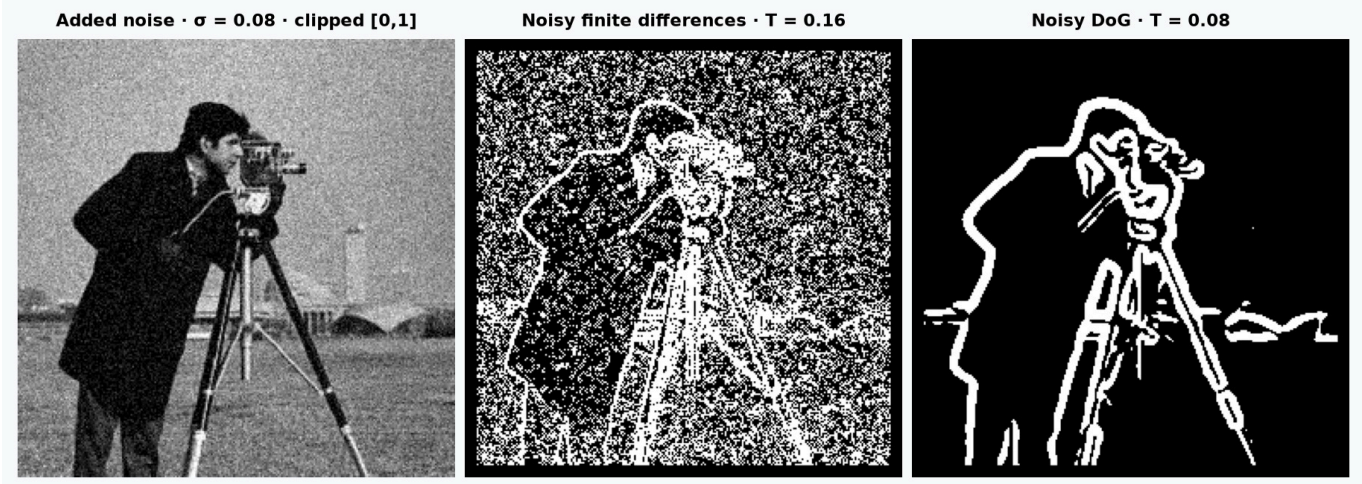
Gaussian and its sampled derivative kernels. A common signed scale is used for the x/y derivative kernels; the Gaussian uses its own nonnegative scale.



Repeating the finite-difference procedure after Gaussian smoothing: signed horizontal and vertical derivatives share the scale [−0.35,0.35], followed by their magnitude on [0,0.35]. These panels use sequential same-sized convolutions with zero padding; the boundary-equivalence comparison below explains the difference from a single DoG convolution.



Gaussian derivative results, raw/DoG threshold maps, and the difference between finite implementations. Preserving the intermediate halo is necessary for global equality.



Controlled noisy cameraman experiment. Gaussian derivatives reject much of the high-frequency noise that triggers raw finite differences.

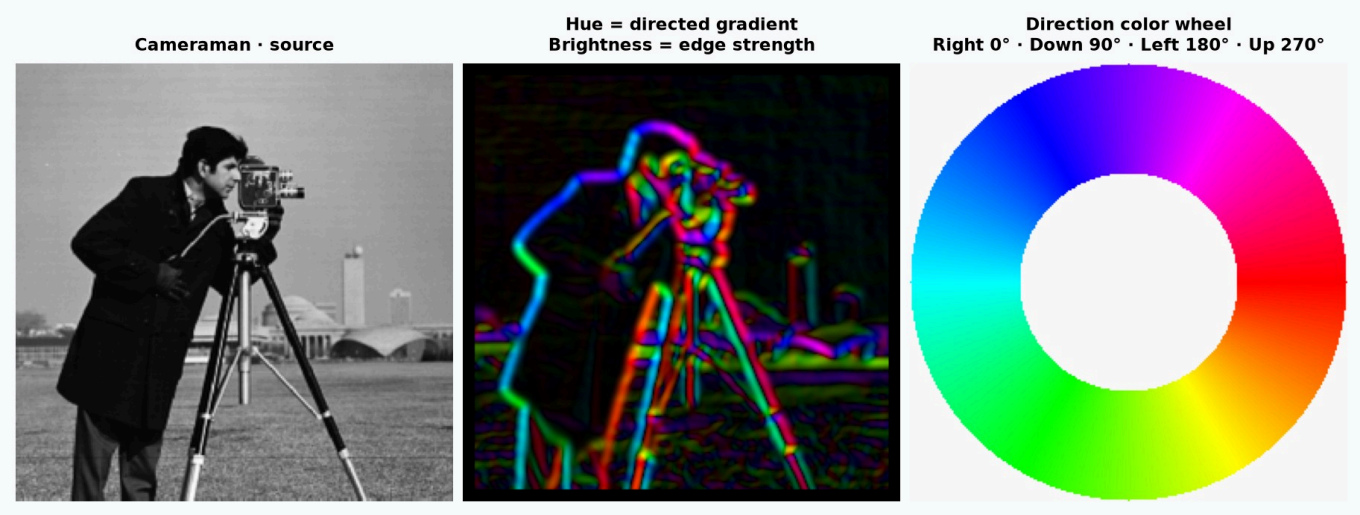
Build Gaussian derivative kernels

```
g1 = cv2.getGaussianKernel(13, 2.0)
G = g1 @ g1.T
Kx = convolve2d(G, DX, mode="full")
Ky = convolve2d(G, DY, mode="full")
gx = convolve2d(image, Kx, mode="same", boundary="fill")
gy = convolve2d(image, Ky, mode="same", boundary="fill")
```

Mandatory extension · Gradient orientations

The orientation implementation calls no arctangent, atan2, or angle routine. It takes r = min(|gy|,|gx|)/max(|gy|,|gx|), then range-reduces r above $\sqrt{2}-1$ using atan(r) = π/4 + atan((r−1)/(r+1)). The reduced argument has magnitude at most $\sqrt{2}-1$, so 18 terms of $z - z^3/3 + z^5/5 - \ldots$ converge accurately. Coordinate swaps and component signs restore all four quadrants; a zero gradient is assigned angle zero and displayed black.

Hue is θ mod 2π divided by 2π; saturation is one. Value is min(magnitude / p99.5, 1)^0.7, with p99.5 = 0.2386, so stronger gradients appear brighter and faint background responses remain subdued. The colors represent directed intensity change, not the tangent direction of the edge. Since image rows increase downward, 90° points down. NumPy atan2 is used only as an independent validation reference: maximum angular error on this figure is 4.44e-16 radians.



HSV orientation visualization computed without built-in angle functions. Direction follows image coordinates; the seven-pixel border is excluded.

Arithmetic-only orientation implementation

```
def orientation_without_angles(gx, gy, terms=18):
    gx, gy = np.broadcast_arrays(np.asarray(gx, float), np.asarray(gy, float))
    if not np.all(np.isfinite(gx)) or not np.all(np.isfinite(gy)):
        raise ValueError('gradient components must be finite')
    if terms < 1:
        raise ValueError('terms must be positive')
    ax, ay = (np.abs(gx), np.abs(gy))
    high, low = (np.maximum(ax, ay), np.minimum(ax, ay))
    ratio = np.divide(low, high, out=np.zeros_like(high), where=high != 0)
    reduce = ratio > np.sqrt(2.0) - 1.0
    z = np.where(reduce, (ratio - 1.0) / (ratio + 1.0), ratio)
    z2 = z * z
    poly = np.zeros_like(z)
    for n in range(terms - 1, -1, -1):
        poly = (-1.0) ** n / (2 * n + 1) + z2 * poly
    small_angle = z * poly + np.where(reduce, np.pi / 4, 0.0)
    angle = np.where(ay > ax, np.pi / 2 - small_angle, small_angle)
    angle = np.where(gx < 0, np.pi - angle, angle)
    angle = np.where(gy < 0, -angle, angle)
    return np.where(high == 0, 0.0, angle)
```

2.1 · Image sharpening

Let $B = G * I$ be the low-pass image and $H = I - B$ its high-frequency residual. Unsharp masking forms $S = I + \alpha H = (1 + \alpha)I - \alpha(G * I)$. Since $I = \delta * I$ and convolution is linear, one convolution suffices: $S = [(1 + \alpha)\delta - \alpha G] * I$. The kernel has unit sum, so a constant image is preserved when the boundary extension also preserves constants.

I use a 13 × 13 Gaussian at $\sigma = 2$ pixels, symmetric boundaries to avoid dark zero-padded borders, and independent float64 RGB-channel processing. All computation keeps signed residuals and unclipped sharpened values until display/export. The figures include the Taj Mahal, an additional original street photograph, and $\alpha = 0.5, 1, 2$. Increasing α strengthens existing edges and texture but also amplifies noise, halos, and values outside the display range.

The single-convolution expression agrees with $I + \alpha(I - G*I)$ at $\alpha = 1$ to a maximum error of 1.78e-15 over these images. High-frequency residuals are displayed as $0.5 + 2H$, where middle gray means zero; they are not independently normalized. The clipped-sample table counts RGB channel values outside [0,1] before clipping, exposing a cost hidden by an attractive display.

For the recovery experiment, I deliberately blur the original street image once, then sharpen that blurred image with the same Gaussian at each tested α. The best tested display result is $\alpha = 2$: clipped RGB MSE changes from 0.001660 after blur to 0.001099 after sharpening, and PSNR changes from 27.80 to 29.59 dB. The parameter was selected against this reference and is not a general optimum.

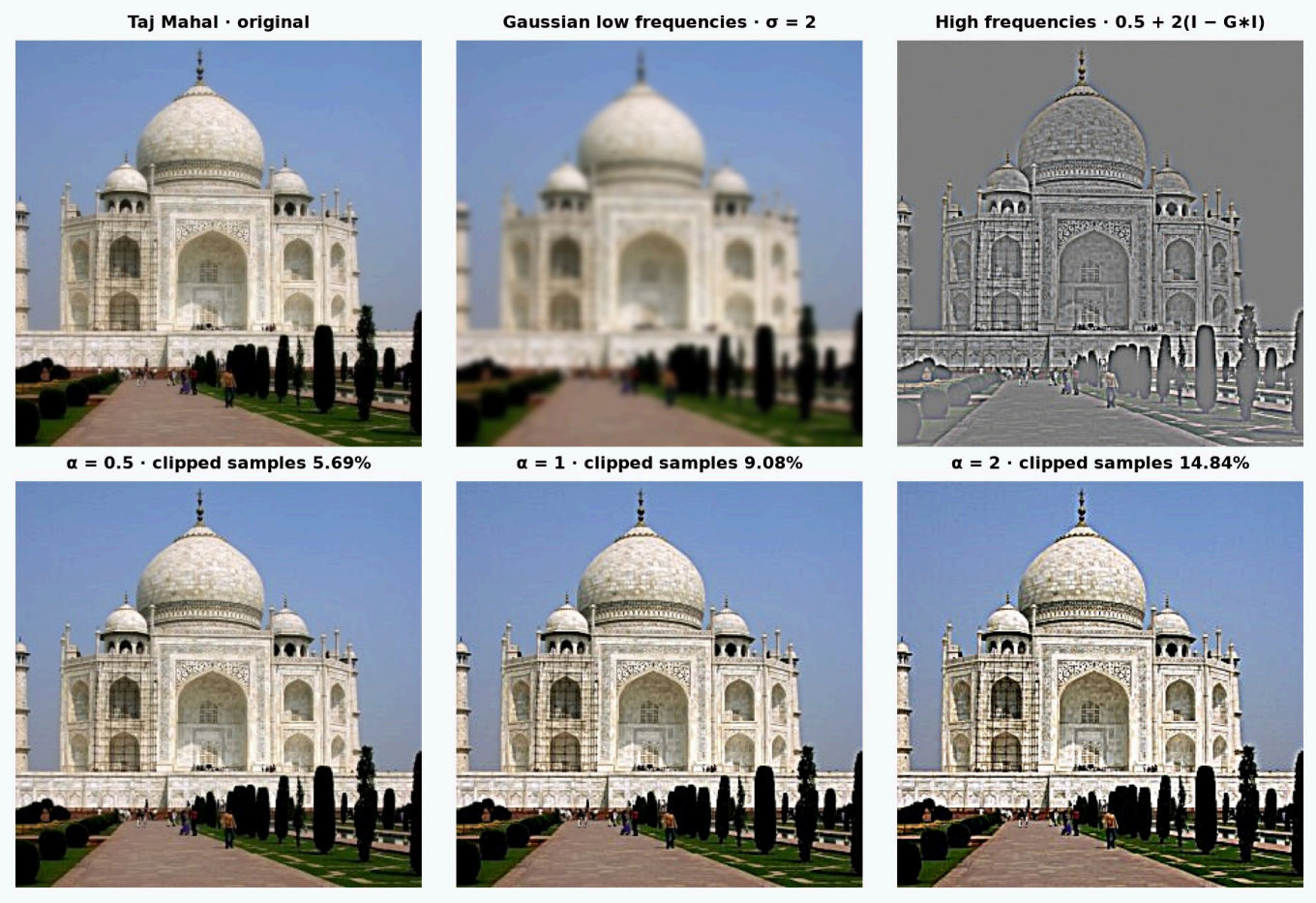
Resharpening makes blurred transitions steeper but does not restore all original detail. If the blur transfer is $G(f)$, the blur-then-unsharp transfer is $G(f)[1 + \alpha - \alpha G(f)]$, which is generally not one. Frequencies reduced nearly to zero cannot be reconstructed by this finite boost, and clipping can discard additional information. The crop, residual map, and nonzero MSE support this limitation; perceptual crispness alone does not establish recovery.

RGB samples outside display range before clipping

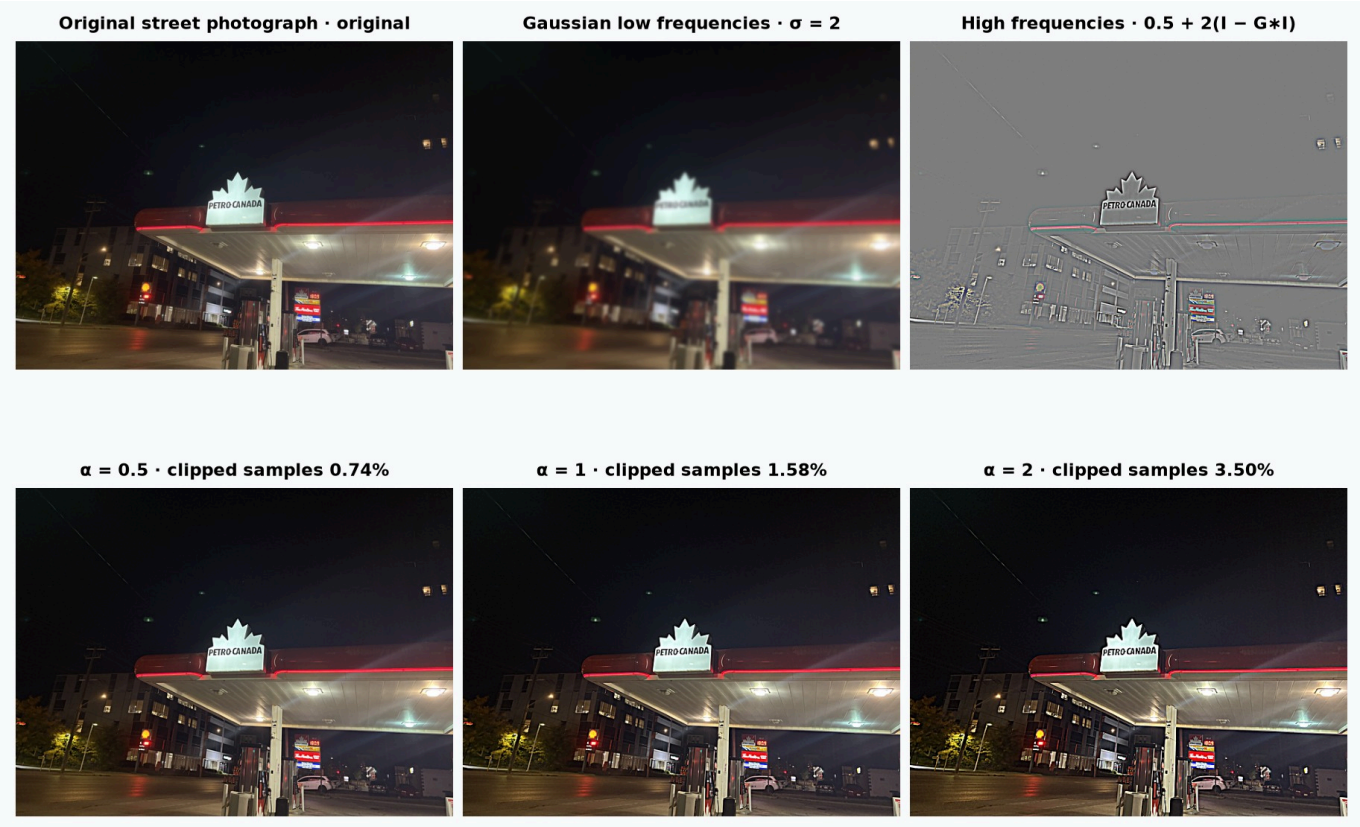
Image	Alpha	Out-of-range samples
Taj Mahal	0.5	5.687%
Taj Mahal	1.0	9.075%
Taj Mahal	2.0	14.845%
Original street photograph	0.5	0.741%
Original street photograph	1.0	1.581%
Original street photograph	2.0	3.504%

Recovery relative to sharp street reference (RGB in [0,1])

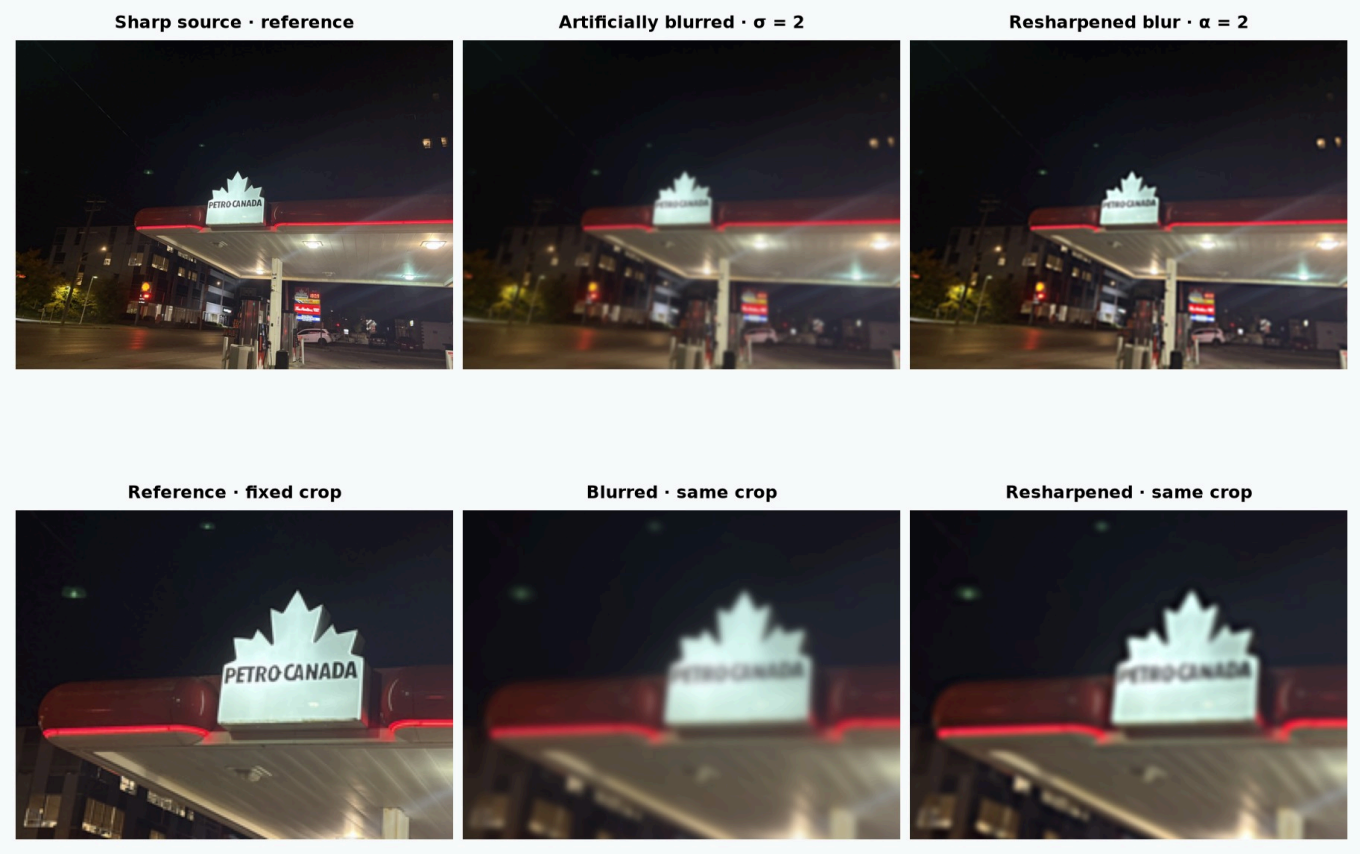
Stage	Alpha	Float MSE	Float PSNR dB	Clipped MSE	Clipped PSNR dB
Original	—	0	∞	0	∞
Blurred	—	0.001660	27.80	0.001660	27.80
Resharpened	0.5	0.001429	28.45	0.001429	28.45
Resharpened	1.0	0.001259	29.00	0.001259	29.00
Resharpened	2.0	0.001101	29.58	0.001099	29.59



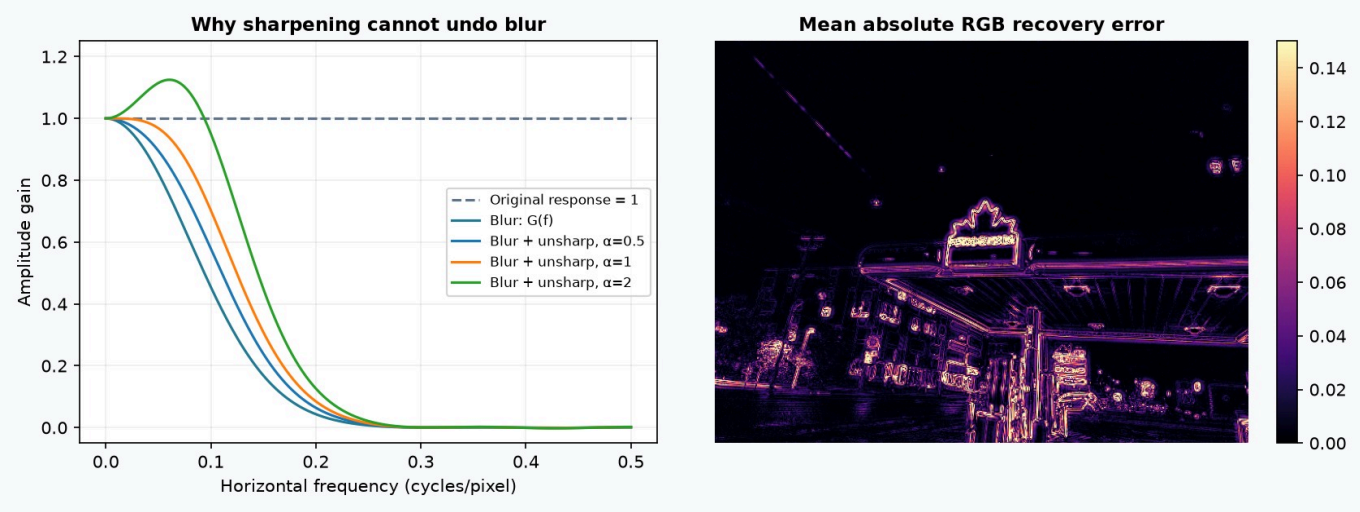
Taj Mahal: original, blurred image, high-frequency residual (mid-gray means zero, amplified 2×), and unsharp results at α = 0.5, 1, 2. RGB channels are filtered separately; final display values are clipped to [0,1].



Original street photograph: original, blurred image, high-frequency residual (mid-gray means zero, amplified 2×), and unsharp results at α = 0.5, 1, 2. RGB channels are filtered separately; final display values are clipped to [0,1].



Blur-and-resharpen experiment on the original street photograph. The lower row enlarges exactly the same fixed region from each output. α = 2 is the best of the three tested settings by clipped RGB MSE against the original; it does not reproduce the source.



Left: horizontal frequency response of the actual discrete Gaussian and the blur-plus-unsharp pipeline. High frequencies that the Gaussian strongly attenuates remain attenuated. Right: residual error against the sharp reference, using the selected recovery setting.

Single-convolution sharpening core (each RGB channel)

```
K = -alpha * G
K[G.shape[0] // 2, G.shape[1] // 2] += 1 + alpha
for c in range(3):
    sharp[..., c] = convolve2d(image[..., c], K, mode="same", boundary="symm")
```

2.2 · Hybrid images

A hybrid adds a blurred source to detail from a second aligned source: H = G_sigmaL * A + gain (B - G_sigmaH * B). Here the people provide the low-frequency image and the cats provide the high-frequency image. The three required examples are Derek + Nutmeg and two custom portrait/cat pairs.

I aligned the eye centers with a two-point similarity transform before filtering. The transform permits rotation, uniform scale and translation, but no face warping. The existing landmarks line up the pupils well, so the refinement changes the frequency separation and gain. Full original coordinates and 2 x 3 transforms are saved with the results.

The initial sigma-low 10, sigma-high 5, gain 1 settings left substantial cat shading at reduced sizes. Moving the high-pass sigma to 1.5 removes more of that broad structure. A gain of 2.2–3 restores fine fur contrast, while sigma-low 6 retains enough human eye and mouth structure to read at smaller sizes. The selected gain is lower for the dense tabby texture than for the lighter portrait cat. These are judgments from the displayed comparisons, not a universally optimal setting.

In the selected astronaut example, the 512-pixel view emphasizes cat fur and eye rims; the astronaut eyes and smile are readable at 128 pixels and clearer at 64. The 256-pixel view still contains both identities. Increasing gain further strengthens fine edges but also clips more highlights; increasing sigma-high leaks broader cat structure back into the reduced view. The six actual trials expose these tradeoffs.

To view the effect, open the largest result at normal size, then increase your viewing distance or compare the smaller versions. Keep the displayed image fixed while moving back. Exact distances depend on screen size, zoom and eyesight, so the pixel-size sequence is a repeatable demonstration rather than a calibrated distance test.

The Gaussian has no sharp cutoff. Its approximate half-amplitude frequency is sqrt(2 ln 2) / (2 pi sigma) cycles per output pixel. For the selected astronaut pair this is 0.0312 for the low pass and 0.1249 for the complementary high pass. The filtering itself is spatial convolution; the Fourier plots diagnose the resulting bands.

All filtering and addition use signed float64 values. Display exports clamp to [0,1] and quantize to 8 bits; the table reports the fraction of scalar channel values clipped. No intermediate band is clipped or contrast-normalized during synthesis. Additional alignment/component views, all other trial grids and the unsuccessful Earth + Moon study are included as supplemental figures.

Selected settings on a 512 x 512 canvas

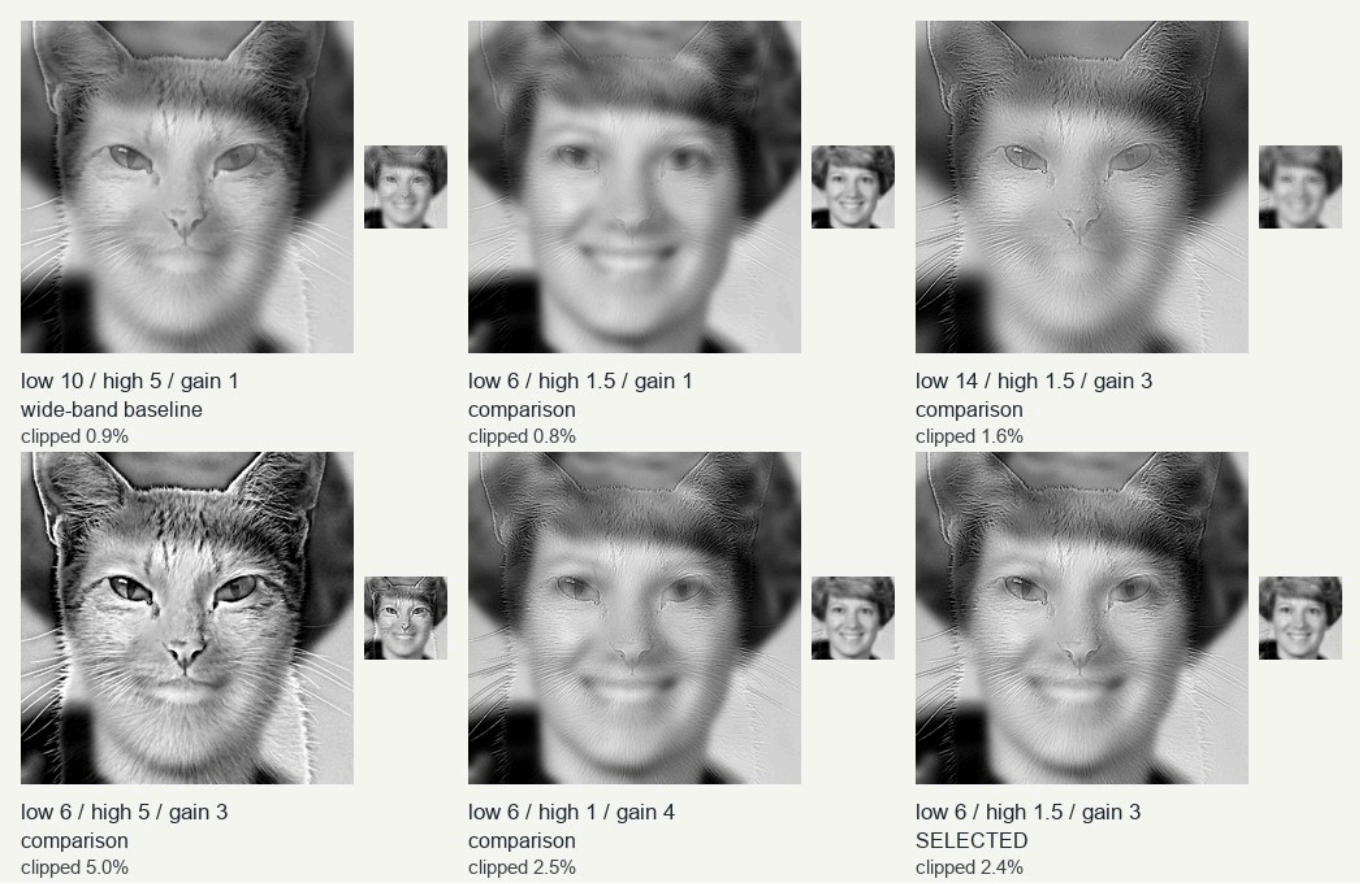
Pair	sigma low (px)	sigma high (px)	gain	values clipped
Derek + Nutmeg	6	1.5	2.5	1.95%
Astronaut + portrait cat	6	1.5	3	2.39%
Obama + tabby	6	1.5	2.2	2.07%

A custom hybrid across viewing scales



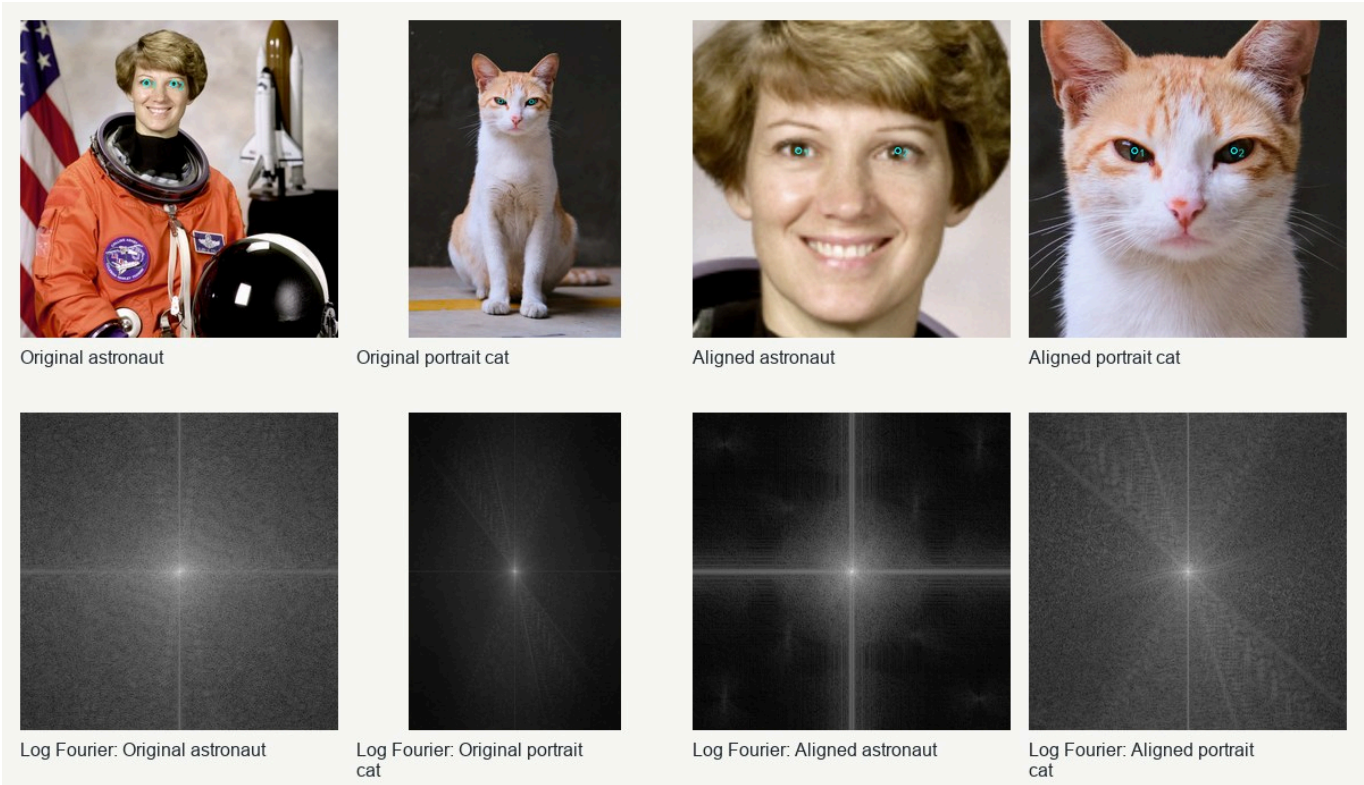
Astronaut + portrait cat: one fixed result at 512, 256, 128 and 64 pixels, reduced with Lanczos. Fine fur and eye rims are visible at the largest size. The astronaut eyes and smile emerge in the smaller views; the intermediate view deliberately exposes the remaining mixture.

Choosing cutoffs and detail gain



Astronaut + portrait cat: every trial is synthesized at 512 x 512, then shown at the same 256-pixel and 64-pixel display sizes. The comparisons vary low-pass width, high-pass width and detail gain; the selected panel uses the exact settings of the final result.

Original sources, alignment and Fourier spectra



Original and aligned inputs for the favorite custom pair. Cyan landmarks are display annotations only; spectra are calculated from the unmarked images. Each spectrum is log(1 + 1000000 |fftshift(fft2(luminance))| / N), using one shared display maximum across both Fourier figures. Division by N normalizes differently sized inputs. No FFT window or mean subtraction is used.

Filtered components and reconstruction



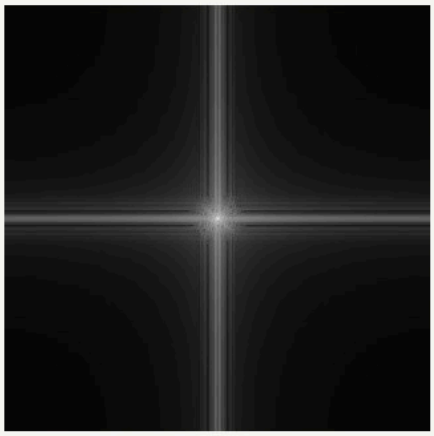
Gaussian low pass



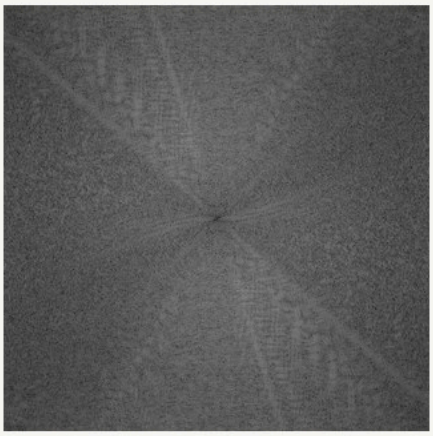
Gain-scaled signed high pass



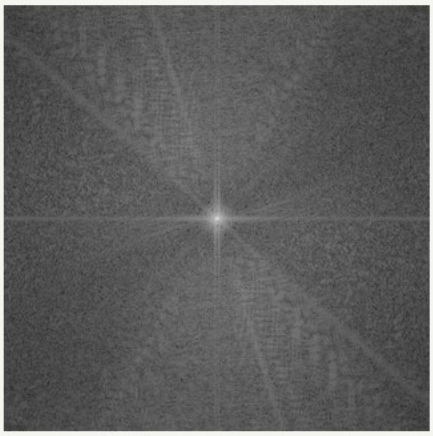
Final hybrid



Log Fourier: Gaussian low pass



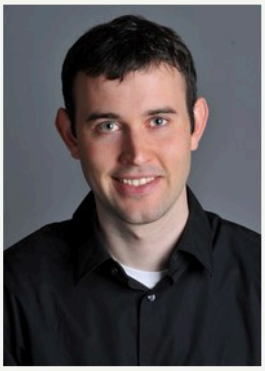
Log Fourier: Gain-scaled signed high pass



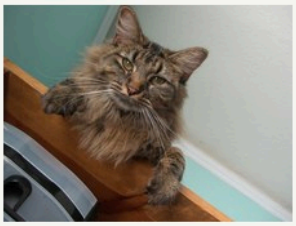
Log Fourier: Final hybrid

The low-pass spectrum is concentrated centrally; the high pass suppresses central energy and preserves fine structure. Their spatial sum produces the final hybrid. The signed high component is displayed as 0.5 + gain H, with clipping only for display. Each spectrum is log(1 + 1000000 |fftshift(fft2(luminance))| / N), using one shared display maximum across both Fourier figures. Division by N normalizes differently sized inputs. No FFT window or mean subtraction is used.

Derek + Nutmeg



Original low source



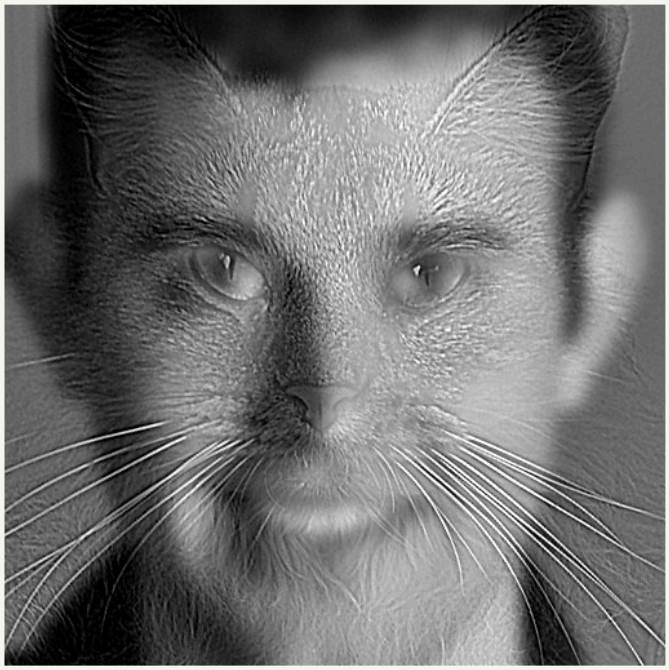
Original high source



128 px



64 px



Hybrid / 512 px

Derek + Nutmeg: complete original sources beside the final hybrid at 512, 128 and 64 pixels. A narrow high-pass band retains whiskers nearby while reducing broad cat shading at smaller scales. Gain 2.5 keeps this detail visible; the distant face remains somewhat mixed at 128 pixels.

Obama + tabby



Original low source



Original high source



128 px



64 px



Hybrid / 512 px

Obama + tabby: complete original sources beside the final hybrid at 512, 128 and 64 pixels. The tabby has dense fur texture, so gain 2.2 is sufficient. A higher gain saturates more fur highlights. The human smile returns at reduced size, although the middle-scale image retains whiskers.

Hybrid synthesis

```
low = gaussian(A, sigma_low)
high = B − gaussian(B, sigma_high)
hybrid = low + gain ∗ high
```

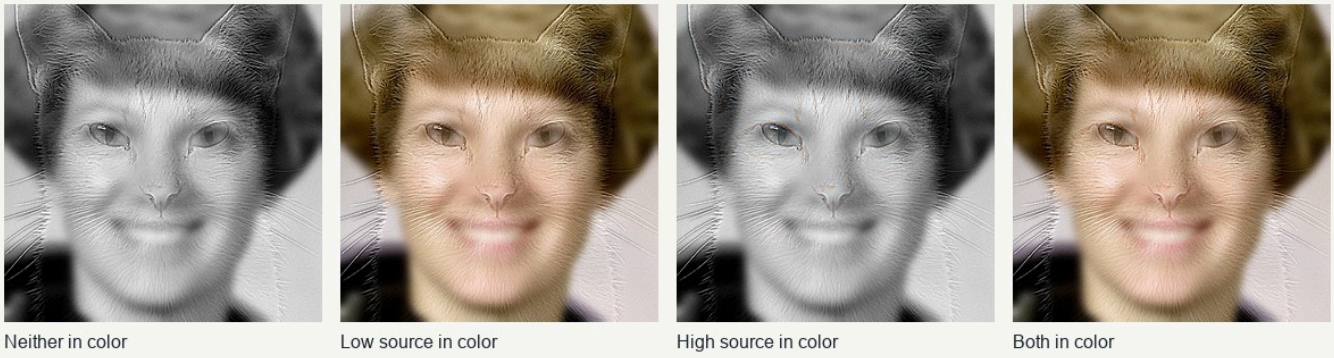
Additional diagnostic figures and parameter trials are available in the online supplement for this section.

Color · Which frequency band benefits?

The four controlled variants put color in neither source, only the low-frequency source, only the high-frequency source, or both. Low-only color preserves broad skin and hair hues associated with the distant astronaut. High-only color contributes colored fine edges but cannot recover the cat's broad coat color after that information has been removed.

For this pair, low-only color is the most coherent color version: its large color regions agree with the distant face, while grayscale fur remains visible nearby. Both-color adds localized competing hues around some cat edges. The grayscale main examples make the frequency-dependent identity easier to judge without a broad color cue.

Color in the low band, high band, or both



The four variants use identical geometry, sigma values and detail gain. Only conversion to luminance changes (weights 0.2126, 0.7152, 0.0722). Broad human skin and hair colors survive the low pass; high-only color is confined to edges and fine cat detail.

2.3 · Gaussian and Laplacian stacks

The apple and orange are the supplied course originals, burt_apple.png and burt_orange.png. Each 740 x 530 input is resized once to 512 x 367 pixels with area interpolation, retaining its aspect ratio to the nearest output pixel. This common working size keeps the experiment efficient without stretching the fruit into a square.

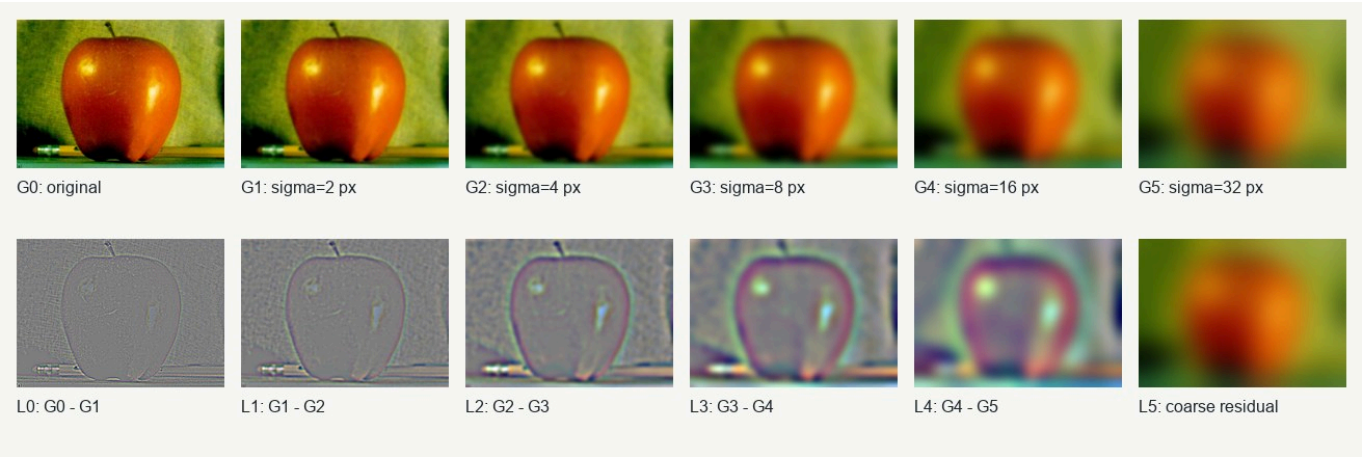
A stack keeps every level at full resolution. I set G0 = I, then blur the original independently at sigma = 2, 4, 8, 16 and 32 pixels. The detail bands are Li = Gi - G(i+1) for i = 0,...,4; L5 = G5 is the coarse residual. Independent blurring makes the effective sigma of each level explicit. No downsampling occurs between stack levels, and no built-in pyramid routine is used.

The residual is essential: summing only the detail bands would recover I - G5 and lose the large-scale brightness and color. With the residual, the sum telescopes exactly to I up to floating-point rounding. The apple and orange reconstruction errors are reported below.

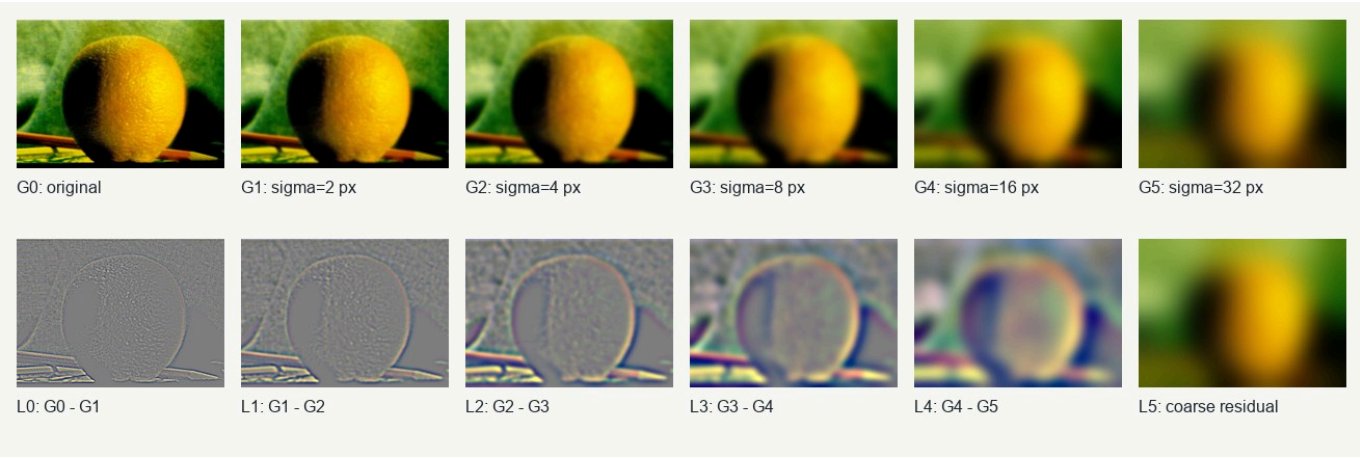
The mask follows the same Gaussian schedule. Each output level is Si = Mi LAi + (1 - Mi) LBi, including the residual level, and the final image is the sum of all Si. A is selected wherever the initial mask is white. This separates fine texture transitions from broad illumination and color transitions.

Stack validation

Invariant	Maximum absolute error
Apple reconstruction	1.110e-16
Orange reconstruction	2.220e-16
Final equals sum of all mixed bands	3.331e-16



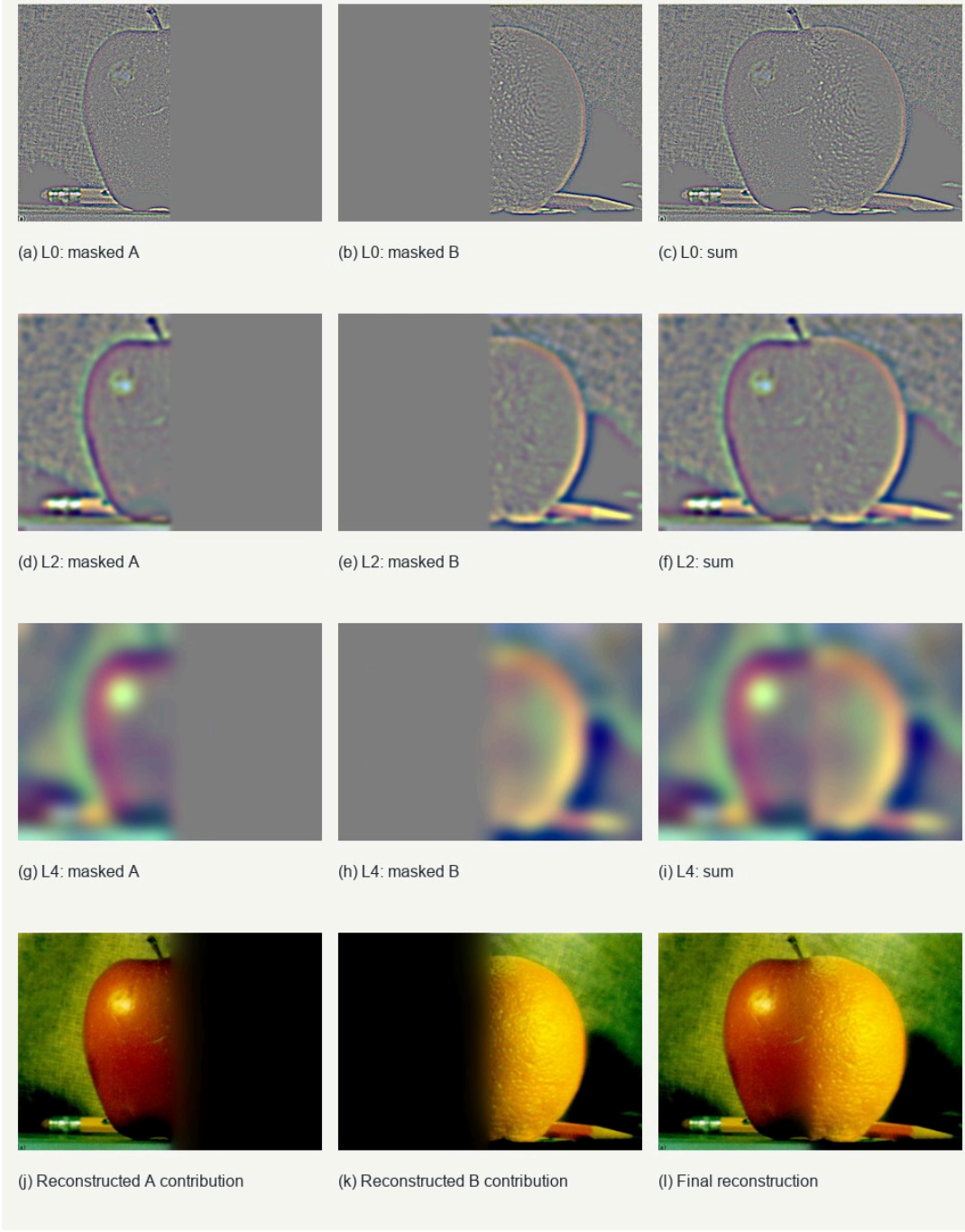
Apple: full-resolution Gaussian stack (top) and Laplacian stack (bottom), including the coarse residual. All six arrays preserve the working resolution of 512 x 367 pixels (width x height). Signed bands use 0.5 + 2.5L for display; the residual uses its actual intensity.



Orange: full-resolution Gaussian stack (top) and Laplacian stack (bottom), including the coarse residual. All six arrays preserve the working resolution of 512 x 367 pixels (width x height). Signed bands use 0.5 + 2.5L for display; the residual uses its actual intensity.



The binary left/right mask is blurred at exactly the Gaussian image-stack scales. Fine bands use a sharp transition; coarser bands mix over progressively wider neighborhoods. White selects apple, black selects orange.



Reproduction of the Figure 3.42 (a)–(l) layout: rows 1–3 show levels 0, 2 and 4, with masked apple, masked orange and their sum in the three columns. Row 4 reconstructs each contribution and the final oraple by summing ALL five detail bands plus the coarse residual. Detail displays share a symmetric contrast scale across each row; row 4 is shown in actual [0,1] intensity.

Explicit stacks and reconstruction

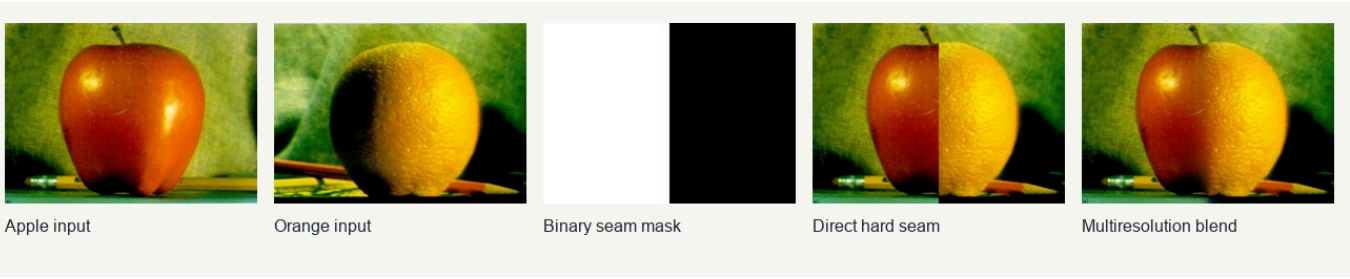
```
G = [I] + [gaussian(I, 2 * 2**i) for i in range(5)]
L = [G[i] - G[i+1] for i in range(5)] + [G[-1]]
S = [M[i] * LA[i] + (1-M[i]) * LB[i] for i in range(6)]
result = sum(S)
```

2.4 · Multiresolution blending

The standard apple/orange blend uses the apple on the left and the orange on the right. A direct binary cut preserves a conspicuous vertical brightness/color discontinuity; mixing corresponding Laplacian bands with Gaussian mask weights spreads that transition according to scale while retaining local texture.

For these inputs, the mean absolute RGB difference across the two columns adjoining the seam decreases from 0.1716 to 0.0279. This local diagnostic supports the visible seam comparison; it is not a general perceptual-quality score.

The seam measurement is computed on the unclipped floating-point reconstruction. For display export only, values are clamped to [0,1]; 2.35% of scalar RGB values require this clamp. No clipping occurs within the stack or before reconstruction.



Course-provided inputs, selection mask, hard-cut baseline and full-color multiresolution oraple. Both baselines use the same aspect-preserving 512 x 367 input images and the same initial mask; the seam is between columns 255 and 256 (zero-based).

Two custom color blends · Irregular selections

Tabby disguise transfers the tabby's central face onto the orange-and-white cat while retaining the destination's ears, whiskers and chest. An irregular 18-vertex polygon follows the forehead and cheeks. Eye-center alignment puts the face texture in a plausible position. The hard cut exposes the polygon immediately; multiscale blending softens the coat-color transition while keeping the transferred eyes and fur crisp.

One world, two surfaces is an imaginary globe: Earth's oceans and clouds transition into lunar craters along an S-shaped boundary. Ellipses fitted to the largest source contours above luminance 0.04 are mapped to a shared circle. This affine registration corrects approximately 1% anisotropy in the photographed Earth outline; the Moon needs almost uniform scaling. Thresholding only measures geometry and does not mask or recolor either photograph. Their shared round outline supports this composite even though their textures did not produce a convincing frequency-switching hybrid. This is an artistic surface fusion, not an astronomical depiction.

Both examples use the entire registered source photographs. The background is never copied into the other source before filtering. Hard cut, single-scale feathering and multiscale blending receive identical arrays and the same binary mask. The feather baseline uses the coarsest Gaussian mask on the full images: its wide transition mixes competing fine details as well as broad colors.

The cat uses five detail bands (sigma = 2, 4, 8, 16, 32 pixels); the globe uses six (2, 4, 8, 16, 32, 64 pixels). Both retain their coarse residual. Supplementary sweeps test four, five and six bands without changing other inputs. For the cat, 16 pixels leaves a more conspicuous facial patch; 32 softens the coat boundary without the broader tonal spill of 64. The globe benefits from 64 pixels because its brightness mismatch spans a large fraction of the disk.

Compared with broad feathering, the cat's cheek and muzzle retain more definite tabby detail; feathering mixes both faces over a broad transition. At the globe's boundary, clouds and crater detail remain more localized in the multiscale result. The boundary-jump values below measure adjacent RGB differences across

the original binary contour. Feathering gives still smaller jumps because it mixes more image detail; minimizing this number would not select the most convincing photograph. It is a local seam diagnostic, not a perceptual score or proof of physical realism.

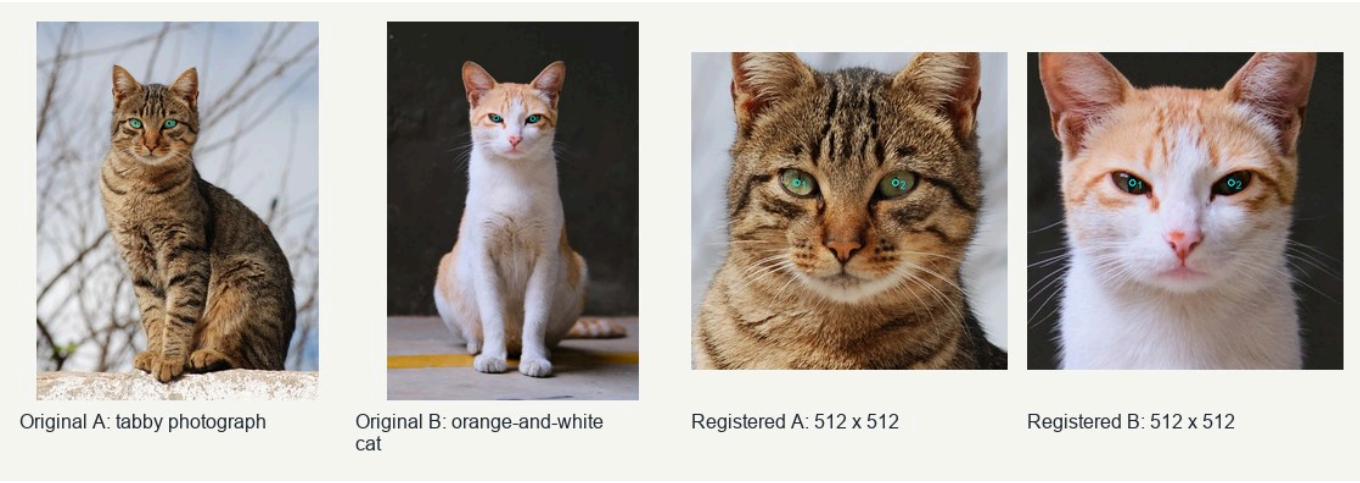
Color helps both concepts. The warm tabby patch remains distinct from white fur and orange ears; grayscale makes these coat regions less distinct. The globe's blue oceans contrast with the nearly neutral Moon and make the two surfaces easier to identify. I prefer RGB for the final composites; grayscale controls show that seam smoothing also works without this color cue.

Multiresolution blending cannot fix anatomical or surface mismatches. The cats still differ slightly in muzzle shape and fur direction, and lunar features cannot continue Earth's clouds naturally. A small transition remains where the globe seam meets the outer limb. These limitations remain visible in the unretouched results and matched close-ups. Processing stays signed float64; only final image display is clipped to [0,1].

Custom-blend settings and local boundary diagnostic

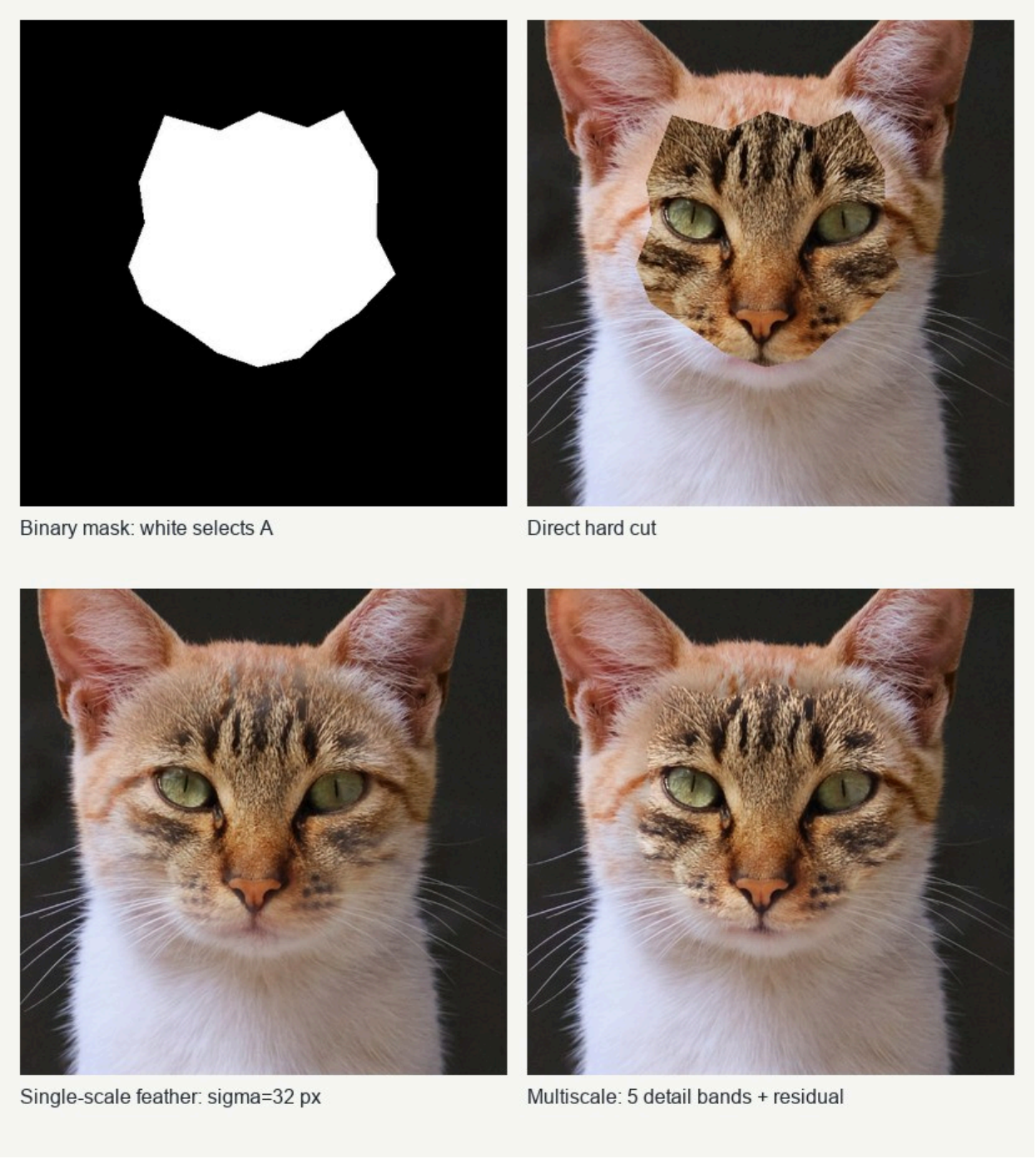
Blend	Mask	Bands	Max sigma	Hard jump	Feather jump	Stack jump	Clipped
Tabby disguise	polygon	5	32	0.2633	0.0297	0.0658	0.00%
One world, two surfaces	sinusoidal	6	64	0.2718	0.0422	0.0808	4.92%

Tabby disguise · Inputs and registration



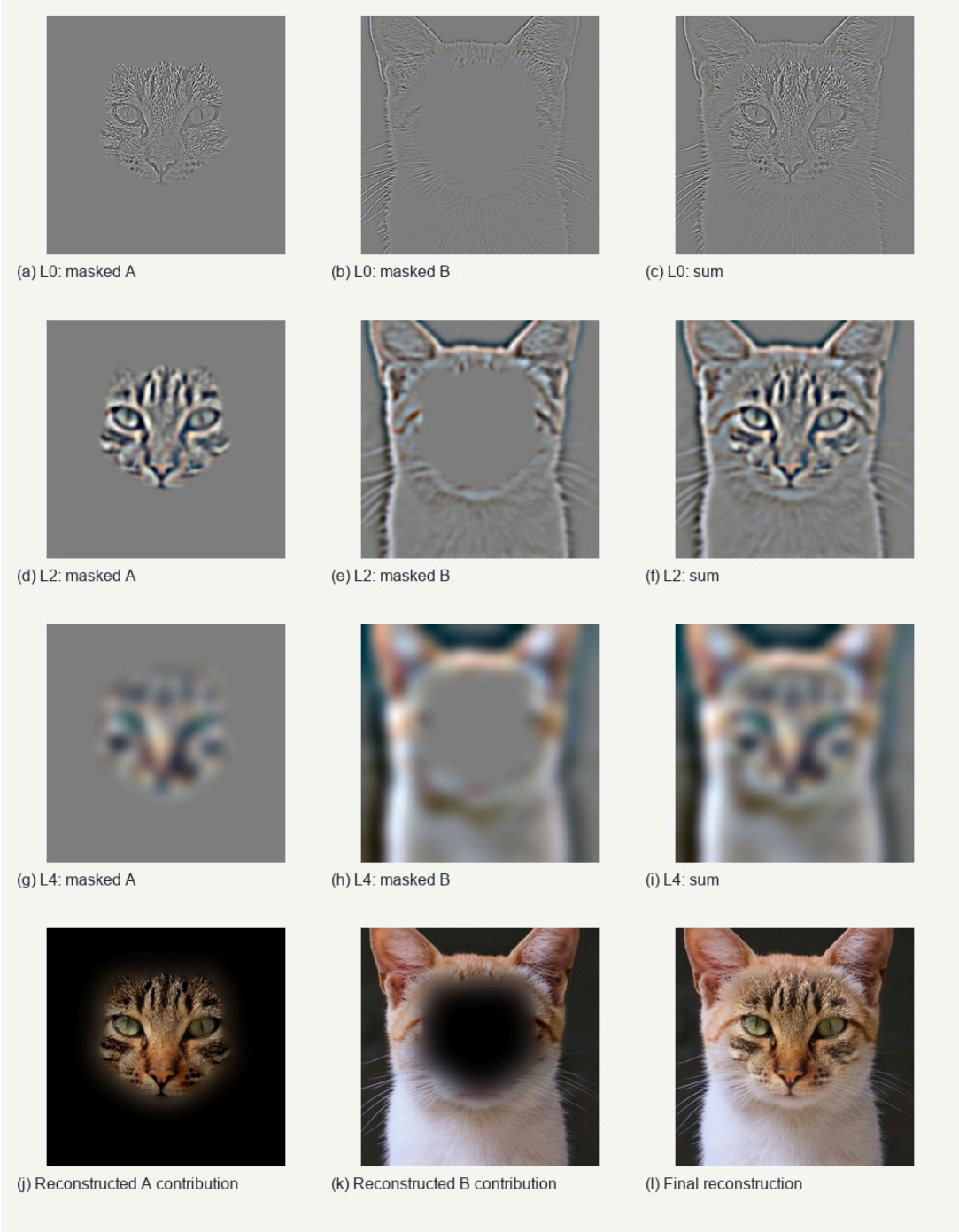
Tabby disguise: original photographs and complete registered inputs. Eye centers align the two faces. Cyan marks show eye centers and their shared destination. The cat inputs use similarity transforms (uniform scale, rotation and translation). Both methods use bilinear interpolation and replicated borders. There is no exposure matching, color correction, background replacement or retouching. Every baseline receives exactly these registered arrays. Tabby photo: Joaquim Alves Gaspar (Alvesgaspar), CC BY-SA 3.0. Orange-and-white cat: Himanshu Choudhary, Unsplash License. The adapted composite and tabby-derived figures are distributed under CC BY-SA 3.0.

Tabby disguise · Mask and fair baselines



Tabby disguise: controlled baselines using the same alignment and binary selection. Feathering applies the stack's coarsest Gaussian mask to every image frequency; the multiscale result uses the corresponding mask at each Laplacian band. The multiscale result softens the forehead/cheek boundary while preserving the tabby eyes and fur. Broad feathering mixes competing facial detail near the cheeks and muzzle. Tabby photo: Joaquim Alves Gaspar (Alvesgaspar), CC BY-SA 3.0. Orange-and-white cat: Himanshu Choudhary, Unsplash License. The adapted composite and tabby-derived figures are distributed under CC BY-SA 3.0.

Favorite blend · Twelve-panel multiscale process



Favorite blend, Tabby disguise: rows 1–3 show masked A, masked B and their sum at Laplacian levels 0, 2 and 4. The final row reconstructs each contribution and the result using ALL detail bands plus the coarse residual, following Burt–Adelson Figure 10. Each detail row uses one shared symmetric contrast scale; reconstructed rows retain actual intensity. Tabby photo: Joaquim Alves Gaspar (Alvesgaspar), CC BY-SA 3.0. Orange-and-white cat: Himanshu Choudhary, Unsplash License. The adapted composite and tabby-derived figures are distributed under CC BY-SA 3.0.

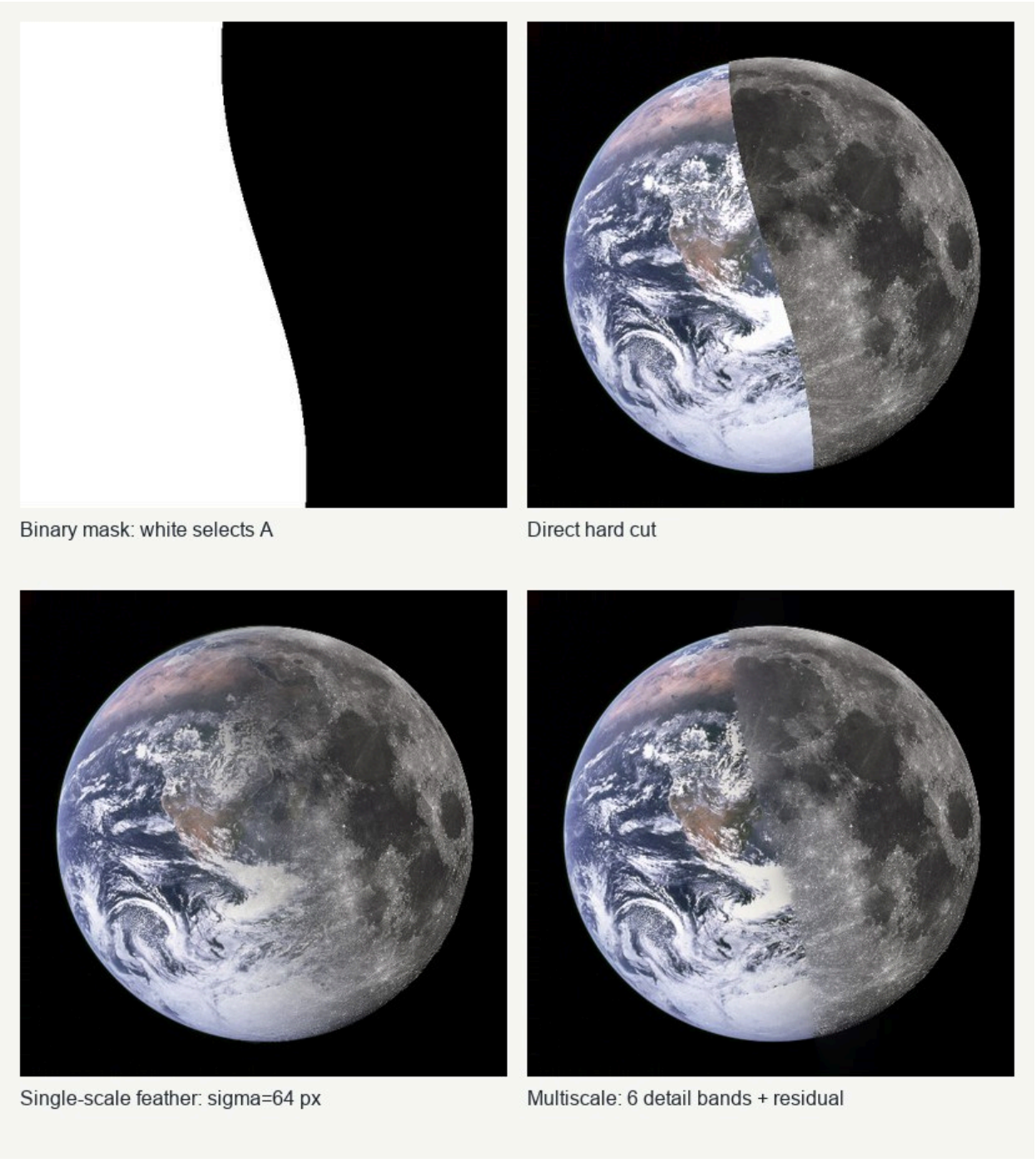
One world, two surfaces · Inputs and registration



Original A: NASA Blue Marble | Original B: Gregory Revera's Moon | Registered A: 512 x 512 | Registered B: 512 x 512

One world, two surfaces: original photographs and complete registered inputs. Measured disk ellipses are mapped to a common circle of radius 220 pixels centered at (256,256). Earth needs a small anisotropic correction; the Moon's measured outline is almost circular. Cyan outlines show fitted source ellipses and the common target circle. The largest external contour above luminance 0.04 is fitted with cv2.fitEllipse, then mapped affinely to a circle. The differential axis scaling is 1.06% for Earth and 0.03% for the Moon. Thresholding measures geometry only; the warp samples the original RGB pixels. Both methods use bilinear interpolation and replicated borders. There is no exposure matching, color correction, background replacement or retouching. Every baseline receives exactly these registered arrays. Earth: NASA/Apollo 17, public domain. Moon: Gregory H. Revera, CC BY-SA 3.0. The adapted composite and Moon-derived figures are distributed under CC BY-SA 3.0.

One world, two surfaces · Mask and fair baselines



One world, two surfaces: controlled baselines using the same alignment and binary selection. Feathering applies the stack's coarsest Gaussian mask to every image frequency; the multiscale result uses the corresponding mask at each Laplacian band. A broad brightness jump becomes a continuous transition from Earth clouds to lunar surface detail; the fine cloud and crater structures remain more localized than in the broad feather mixture. The seam is S-shaped rather than straight. Earth: NASA/Apollo 17, public domain. Moon: Gregory H. Revera, CC BY-SA 3.0. The adapted composite and Moon-derived figures are distributed under CC BY-SA 3.0.

Color experiment · Luminance and RGB



Tabby disguise / luminance



Tabby disguise / RGB



One world, two surfaces / luminance



One world, two surfaces / RGB

Controlled luminance-versus-RGB comparison. Registration, masks and all stack settings remain fixed. Luminance is 0.2126R + 0.7152G + 0.0722B; RGB channels are blended independently. Tabby and Moon derivatives in this figure are distributed under CC BY-SA 3.0; source credits are listed below.

Additional diagnostic figures and parameter trials are available in the online supplement for this section.

What I learned

The most important thing I learned is that choosing a spatial scale changes which image structures matter. A derivative exposes both edges and noise; smoothing selects broader contours. A hybrid lets fine texture and broad shape describe different subjects. In blending, narrow transitions preserve detail while wider transitions reconcile illumination. The new cat and planetary examples make that distinction clear against hard-cut and feathered baselines. I also learned to separate numerical correctness from visual success: exact reconstruction cannot repair mismatched anatomy or surface detail. Signed floating-point calculations, explicit boundary conditions, and honest display/clipping descriptions make these judgments reproducible.

Reproduction and image credits

The source package includes the input images, explicit algorithms, saved parameters, meaningful numerical tests, measured results, and report builder. The custom examples use sourced photographs that can be retained as the final examples or replaced. The self-portrait is from the supplied Assignment 1 photographs. The apple and orange inputs are the original PNGs from the supplied Part 2 image package; the supplied Taj Mahal and hybrid photographs match the earlier inputs exactly. Source records, complete rights statements, and hashes are included in the image manifest.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m unittest discover -s tests -v
python run_filters.py
python run_frequencies.py
python build_report.py
python -m http.server 8000 --directory site
```

See README.md for image replacement, environment details, and PDF export. Measured errors, timings, and experiment settings are available in measurements.json.

Image	Credit	Rights / provenance	Link
Derek portrait	Derek Hoiem / course materials	Course sample; see note below.	Source
Nutmeg the cat	Derek Hoiem / course materials	Course sample; see note below.	Source
Taj Mahal	Course materials; individual photographer not specified	Course sample; see note below.	Source
Classic cameraman	Course materials; individual photographer not specified	Course sample; see note below.	Source
Apple (supplied Burt-Adelson original)	Peter J. Burt and Edward H. Adelson / course materials	Course sample; see note below.	Supplied Part 2 image package
Orange (supplied Burt-Adelson original)	Peter J. Burt and Edward H. Adelson / course materials	Course sample; see note below.	Supplied Part 2 image package
Moksh portrait	User-supplied photograph	User-provided photograph.	Supplied photograph
Street photograph	User-supplied photograph	User-provided photograph.	Supplied photograph
Astronaut Eileen Collins portrait	NASA	Public domain (NASA photograph)	Source License
The Blue Marble / Earth seen from Apollo 17	NASA / Apollo 17 crew	Public domain (U.S. government work)	Source License
Full Moon, 22 October 2010	Gregory H. Revera	CC BY-SA 3.0	Source License
Barack Obama official portrait, December 2012	Pete Souza / White House	Public domain (U.S. government work)	Source
Young male tabby cat, Portugal, November 2010	Joaquim Alves Gaspar (Alvesgaspar)	CC BY-SA 3.0	Source License
Front-facing orange-and-white cat, Bengaluru	Himanshu Choudhary (@aalochak)	Unsplash License	Source License

Course samples are supplied for this assignment; no general open license is asserted. Personal photographs are user-provided. Adapted figures containing the Moon photograph by Gregory H. Revera or the tabby photograph by Alvesgaspar remain under CC BY-SA 3.0, with full attribution recorded in the image manifest.

References

1. A. Oliva, A. Torralba, and P. G. Schyns. Hybrid images. ACM Transactions on Graphics 25(3), 527–532, 2006.
2. P. J. Burt and E. H. Adelson. A multiresolution spline with application to image mosaics. ACM Transactions on Graphics 2(4), 217–236, 1983.
3. R. Szeliski. Computer Vision: Algorithms and Applications, second edition. Figure 3.42.
4. SYDE 671 Assignment 2, “Fun with Filters and Frequencies.” Required experiments and extensions follow the supplied handout, adapted from Berkeley CS180.